

Deep Q-Network (DQN) et Rainbow DQN

De la théorie à l'état de l'art

Hautot Julien

Cours de Reinforcement Learning

19 novembre 2025

Sommaire

1 Fondamentaux et Théorie

2 Architecture DQN

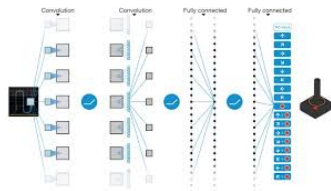
3 Rainbow DQN

Introduction

Deep Q-Network (DQN)

Le DQN combine le **Q-learning** et les **réseaux de neurones profonds** pour approximer la fonction d'action-valeur $Q(s, a)$.

Introduit par DeepMind (2013/2015), il a permis de maîtriser les jeux Atari directement à partir des pixels.



Source : DeepMind

Rappel : Q-Learning

- **Objectif** : Approximer $Q(s, a)$, la récompense cumulée attendue.
- **Mise à jour (Bellman)** :

Formule de mise à jour

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[\underbrace{r + \gamma \max_{a'} Q(s', a')}_{\text{Cible TD}} - Q(s, a) \right]$$

- **Limite** : Le Q-learning tabulaire ne gère pas les grands espaces d'états (ex : image 84×84 pixels).

Principe fondamental : Équation de Bellman

L'équation d'optimalité de Bellman est une propriété structurelle :

$$Q^*(s, a) = \mathbb{E}[r + \gamma \max_{a'} Q^*(s', a')]$$

La valeur optimale se décompose en deux termes :

- 1 La récompense immédiate r
- 2 La valeur optimale future actualisée $\gamma \max Q^*$

Intuition

L'équation de Bellman n'est pas "déduite" : c'est une définition de la cohérence. Un Q optimal est cohérent avec sa propre vision du futur.

La boucle d'apprentissage :

- Si Q est correct $\rightarrow$ il vérifie l'équation.
- Si Q ne la vérifie pas $\rightarrow$ il n'est pas optimal.

$\Rightarrow$ On force progressivement Q à devenir son *"meilleur prédicteur de lui-même"*.

Le Bootstrapping

Le bootstrapping, c'est utiliser une estimation pour en mettre à jour une autre.

$$\text{Cible} = r + \gamma \max_{a'} Q(s', a'; \theta)$$

Ce $\max Q$ est **une prédiction du réseau**, pas une vérité externe.

Analogie

C'est comme apprendre à marcher en regardant ses propres pas précédents, sans professeur pour dire "c'est bien" à chaque seconde.

Compromis :

- + Accélère l'apprentissage (réutilisation de l'info).
- Introduit de l'instabilité (biais de confirmation).

Passage au DQN : Les défis

Remplacer la table Q par un réseau de neurones $Q(s, a; \theta)$.

Problèmes majeurs des réseaux naïfs

- ❶ **Corrélation des données** : Les états successifs (s_t, s_{t+1}) sont très corrélés.
- ❷ **Cible mouvante** : On met à jour Q en utilisant... Q . Si Q change, la cible change aussi $\rightarrow$ Oscillation.

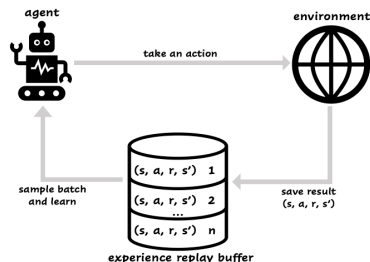
Solution 1 : Experience Replay

Concept :

- Stocker les transitions (s, a, r, s', d) dans un *buffer* circulaire.
- Apprendre sur des **mini-batches** aléatoires.

Bénéfices :

- Rompt la corrélation temporelle.
- Réutilise les données (efficacité).



Solution 2 : Target Network

On utilise deux réseaux pour stabiliser la "cible mouvante".

- **Réseau Online** (Q_θ) : Sélectionne l'action, mis à jour à chaque pas.
- **Réseau Target** (Q_{θ^-}) : Calcule la valeur cible, figé temporairement.

$$y = r + \gamma \max_{a'} Q(s', a'; \theta^-)$$

Mise à jour : $\theta^- \leftarrow \theta$ tous les C pas (Hard update) ou par moyenne glissante (Soft update).

Algorithme DQN complet

- ① Initialiser réseaux (Q, Q^-) et Buffer D .
- ② **Pour chaque épisode :**
 - Observer s .
 - Choisir a avec politique ϵ -greedy (Exploration).
 - Exécuter a , observer r, s' .
 - Stocker (s, a, r, s') dans D .
 - **Apprentissage :**
 - Échantillonner un batch aléatoire de D .
 - Calculer la cible y avec Q^- .
 - Descente de gradient sur $(y - Q(s, a))^2$.
 - Mettre à jour Q^- périodiquement.

Rainbow DQN : Vue d'ensemble

Rainbow agrège 6 améliorations majeures du DQN standard.

❶ **Double DQN**

Réduit la surestimation.

❷ **Dueling Networks**

Sépare Valeur / Avantage.

❸ **Prioritized Replay**

Focus sur les surprises.

❹ **Multi-step Learning**

Vision plus loin.

❺ **Distributional RL**

Apprend la distribution.

❻ **Noisy Networks**

Exploration intégrée.

L'union fait la force : Rainbow atteint l'état de l'art sur Atari.

1. Double DQN (DDQN)

Problème : $\max_{a'} Q(s', a')$ surestime systématiquement les valeurs (biais positif).

Solution : Découpler le choix et l'évaluation.

- **Sélection** de l'action optimale $\rightarrow$ Réseau Online θ .
- **Évaluation** de sa valeur $\rightarrow$ Réseau Target θ^- .

Nouvelle Cible

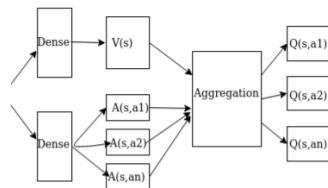
$$Y^{DDQN} = r + \gamma Q(s', \underbrace{\arg \max_{a'} Q(s', a'; \theta)}_{\text{Choix}}; \theta^-)$$

2. Dueling Network Architecture

Idée : Savoir si un état est "bon" ($V(s)$) est parfois plus important que de connaître l'action exacte.

$$Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{|\mathcal{A}|} \sum A(s, a') \right)$$

- Deux têtes de sortie partagent les couches de convolution.
- Accélère la convergence.



Architecture à deux têtes

3. Prioritized Experience Replay (PER)

Problème : L'échantillonnage uniforme est inefficace. Pourquoi revoir ce qu'on sait déjà ?

Solution : Favoriser les transitions avec une forte erreur TD (δ).

$$P(i) = \frac{|\delta_i|^\alpha}{\sum_k |\delta_k|^\alpha}$$

Attention au biais

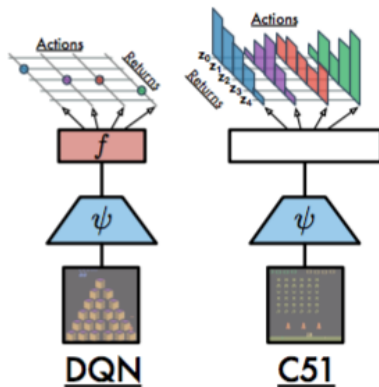
Changer la fréquence d'échantillonnage biaise l'espérance. Il faut corriger avec des poids d'importance (*IS Weights*) :

$$w_i = \left(\frac{1}{N \cdot P(i)} \right)^\beta$$

4. Distributional RL (C51)

Changement de paradigme : Au lieu de prédire une moyenne (l'espérance), on prédit la **distribution de probabilité** des retours futurs.

- Discrétisation du support en 51 atomes.
- Le réseau sort un vecteur softmax (probabilités).
- Capture la variance et la multimodalité (risques).



5. N-Step Learning

Un compromis entre Monte-Carlo (attendre la fin) et TD(0) (1 pas).

$$G_{t:t+n} = r_t + \gamma r_{t+1} + \dots + \gamma^{n-1} r_{t+n-1} + \gamma^n \max_a Q(s_{t+n}, a)$$

Avantages :

- Propagation plus rapide de la récompense.
- Moins de biais que le 1-step, moins de variance que Monte-Carlo.

6. Noisy Networks

Objectif : Remplacer l'exploration ϵ -greedy (aléatoire et rigide) par une exploration déterministe apprise.

Méthode : Ajouter du bruit directement dans les poids du réseau.

$$Y = (\mu_w + \sigma_w \odot \epsilon)X + (\mu_b + \sigma_b \odot \epsilon)$$

- Le réseau apprend à réduire σ (le bruit) quand il devient confiant.
- L'exploration est cohérente au sein d'un même épisode.

Conclusion

Synthèse

Rainbow DQN montre que la combinaison intelligente de modules indépendants (réduction de biais, meilleure exploration, meilleure modélisation) crée un agent surhumain.

Et après ?

- **Ape-X** : Distribution de l'apprentissage sur plusieurs machines.
- **R2D2** : Ajout de mémoire récurrente (LSTM) pour les environnements partiellement observables.